Spring Boot

Complete In-Depth Guide

Spring Boot Fundamentals • Auto-Configuration • Web Applications

REST APIs • Spring MVC • Actuator • Configuration In-Depth

With Clear Definitions, Usage Patterns & Full Code Examples

Table of Contents

CHAPTER 1

Introduction to Spring Boot

Convention over Configuration – Production-ready apps in minutes

1.1 What is Spring Boot?

Spring Boot

Spring Boot is an opinionated, convention-over-configuration framework built on top of the Spring Framework. It eliminates the need for extensive XML or Java configuration by providing sensible defaults, auto-configuration, and embedded servers (Tomcat, Jetty, Undertow). The goal is to let developers start writing business logic immediately without spending time on boilerplate setup.

- **Embedded Server** – Ships with Tomcat by default; no external deployment needed.
- **Starter POMs** – Curated dependency bundles (spring-boot-starter-web, -data-jpa, etc.).
- **Auto-Configuration** – Automatically configures Spring beans based on classpath and properties.
- **Actuator** – Built-in production monitoring endpoints (health, metrics, info).
- **Spring Initializr** – Web-based project generator at start.spring.io.
- **No code generation** – No XML required; everything is Java/annotations.

1.2 Spring Framework vs Spring Boot

Aspect	Spring Framework	Spring Boot
Configuration	Manual XML or Java @Configuration	Auto-configured; minimal setup
Web Server	Deploy WAR to external Tomcat/JBoss	Embedded Tomcat; run as JAR
Dependencies	Add individual Spring module JARs	spring-boot-starter-* bundles
Startup	Needs ApplicationContext bootstrap code	@SpringBootApplication + main()
Properties	Manual PropertySource setup	application.properties auto-loaded
Production	Manual setup for health, metrics	Actuator built in
Testing	Manual context wiring	@SpringBootTest, @WebMvcTest

1.3 Spring Boot Starters

Starter POM

A starter is a pre-packaged set of Maven/Gradle dependencies for a specific concern. Adding spring-boot-starter-web automatically brings in Spring MVC, Jackson, Tomcat, Hibernate Validator, and all

required transitive dependencies at compatible versions.

Starter	What it includes
spring-boot-starter-web	Spring MVC, Tomcat, Jackson, Hibernate Validator
spring-boot-starter-data-jpa	Spring Data JPA, Hibernate, JPA API
spring-boot-starter-data-jdbc	Spring JDBC, HikariCP connection pool
spring-boot-starter-security	Spring Security (auth, authorisation)
spring-boot-starter-test	JUnit 5, Mockito, AssertJ, MockMvc
spring-boot-starter-actuator	Production monitoring endpoints
spring-boot-starter-thymeleaf	Thymeleaf template engine for HTML views
spring-boot-starter-mail	JavaMail API for sending emails
spring-boot-starter-cache	Spring Cache abstraction
spring-boot-starter-aop	AspectJ AOP support

1.4 Creating a Project – Spring Initializr

- Visit **start.spring.io** in your browser.
- Choose: Project = Maven, Language = Java, Spring Boot = 3.3.x.
- Fill in Group (com.example), Artifact (myapp), Packaging = Jar, Java = 17.
- Add Dependencies: Spring Web, Spring Data JPA, H2 Database, Spring Boot DevTools.
- Click Generate – download and unzip into your Eclipse/IntelliJ workspace.
- In Eclipse: File → Import → Existing Maven Projects → select the unzipped folder.

pom.xml – Spring Boot Parent + Starters

```
<!-- Minimal pom.xml for a Spring Boot Web project -->
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.3.0</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

```
<dependency>
<groupId>com.h2database</groupId>
<artifactId>h2</artifactId>
<scope>runtime</scope>
</dependency>
</dependencies>
```

1.5 Project Structure

■ Standard Spring Boot Project Layout

```
myapp/
■■■ src/
■ ■■■ main/
■ ■ ■■■ java/com/example/myapp/
■ ■ ■ ■■■ MyappApplication.java ← @SpringBootApplication + main()
■ ■ ■ ■■■ controller/ ← @RestController classes
■ ■ ■ ■■■ service/ ← @Service classes
■ ■ ■ ■■■ repository/ ← @Repository / JpaRepository
■ ■ ■ ■■■ model/ ← @Entity classes
■ ■ ■ ■■■ config/ ← @Configuration classes
■ ■ ■■■ resources/
■ ■ ■■■ application.properties ← Main configuration file
■ ■ ■■■ application-dev.properties ← Dev profile config
■ ■ ■■■ static/ ← CSS, JS, images
■ ■ ■■■ templates/ ← Thymeleaf HTML templates
■ ■■■ test/java/com/example/myapp/
■ ■■■ MyappApplicationTests.java
■■■ pom.xml
```

■ Main Application Class

```
// MyappApplication.java – Entry point
package com.example.myapp;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MyappApplication {
public static void main(String[] args) {
SpringApplication.run(MyappApplication.class, args);
}
}
```

CHAPTER 2

Spring Boot Auto-Configuration

Magic explained – how Spring Boot wires everything automatically

2.1 How Auto-Configuration Works

Auto-Configuration

Auto-configuration is the mechanism by which Spring Boot automatically creates and registers Spring beans based on the JARs on the classpath, the properties you've defined, and any beans you've already declared. It uses @Conditional annotations to apply configuration only when certain conditions are met – for example, only configure a DataSource if a JDBC driver JAR is present and spring.datasource.url is set.

The auto-configuration process:

- **Step 1** – JVM loads all JARs in the classpath.
- **Step 2** – Spring Boot reads META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports (Boot 3.x) from every JAR.
- **Step 3** – Each listed auto-configuration class is evaluated against its @Conditional conditions.
- **Step 4** – If conditions pass, the @Configuration class is applied and its @Bean methods register beans.
- **Step 5** – User-defined beans take priority; auto-configured beans are only created if missing.

> ✔ You can see exactly what auto-configuration was applied and why by running your app with --debug flag or setting debug=true in application.properties. Look for the CONDITIONS EVALUATION REPORT in the console.

2.2 @SpringBootApplication Deep Dive

@SpringBootApplication

@SpringBootApplication is a meta-annotation that combines three annotations into one: @SpringBootConfiguration (marks this as a configuration class), @EnableAutoConfiguration (triggers the auto-configuration mechanism), and @ComponentScan (scans the current package and sub-packages for Spring components).

■ @SpringBootApplication expanded

```
// What @SpringBootApplication expands to:
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@SpringBootConfiguration // same as @Configuration
@EnableAutoConfiguration // triggers auto-config
```

```
@ComponentScan(excludeFilters = {
@Filter(type=ASSIGNABLE_TYPE, value=TypeExcludeFilter.class),
@Filter(type=ASSIGNABLE_TYPE, value=AutoConfigurationExcludeFilter.class)
})
public @interface SpringBootApplication {
// scanBasePackages, exclude, excludeName ...
}
// Usage with custom scan base
@SpringBootApplication(scanBasePackages = "com.example")
public class MyApp {
public static void main(String[] args) {
SpringApplication.run(MyApp.class, args);
}
}
```

2.3 @Conditional Annotations

@Conditional

Auto-configuration classes use @Conditional annotations to guard bean registration. A condition is evaluated at startup; only if it returns true does Spring apply the configuration.

Annotation	Condition – bean is created only if...
@ConditionalOnClass(Foo.class)	Class Foo is present on the classpath
@ConditionalOnMissingClass	A class is NOT on the classpath
@ConditionalOnBean(Foo.class)	A bean of type Foo is already registered
@ConditionalOnMissingBean(Foo.class)	No bean of type Foo is registered yet
@ConditionalOnProperty(name, value)	A property key has a specific value
@ConditionalOnWebApplication	The application is a web application
@ConditionalOnNotWebApplication	The application is NOT a web application
@ConditionalOnResource	A specific resource file exists
@ConditionalOnExpression	A SpEL expression evaluates to true
@ConditionalOnJava	Running on a specific Java version

■ Auto-Config with @Conditional (simplified)

```
// Example: DataSource auto-config (simplified)
@Configuration
@ConditionalOnClass({ DataSource.class, EmbeddedDatabase.class })
@ConditionalOnMissingBean(DataSource.class) // skip if user defined one
@EnableConfigurationProperties(DataSourceProperties.class)
```

```
public class DataSourceAutoConfiguration {

@Bean
@ConditionalOnProperty(name="spring.datasource.url")
public DataSource dataSource(DataSourceProperties props) {
return DataSourceBuilder.create()
.url(props.getUrl())
.username(props.getUsername())
.password(props.getPassword())
.build();
}
}
```

2.4 Disabling Auto-Configuration

■ Disabling Auto-Configuration

```
// Method 1: exclude on @SpringBootApplication
@SpringBootApplication(exclude = {
DataSourceAutoConfiguration.class,
SecurityAutoConfiguration.class
})
public class MyApp { ... }

// Method 2: exclude by class name (when class not available)
@SpringBootApplication(excludeName = {
"org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration"
})
public class MyApp { ... }

// Method 3: application.properties
# spring.autoconfigure.exclude=\
# org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

2.5 Writing a Custom Auto-Configuration

■ Custom Auto-Configuration

```
// Step 1: Create the auto-configuration class
@Configuration
@ConditionalOnClass(SmsService.class)
@ConditionalOnMissingBean(SmsService.class)
@EnableConfigurationProperties(SmsProperties.class)
public class SmsAutoConfiguration {

@Bean
public SmsService smsService(SmsProperties props) {
return new SmsService(props.getApiKey(), props.getSenderId());
}
```

```
}

// Step 2: Create the properties binding class
@ConfigurationProperties(prefix = "sms")
public class SmsProperties {
private String apiKey;
private String senderId;
// getters and setters
}

// Step 3: Register in
// src/main/resources/META-INF/spring/
// org.springframework.boot.autoconfigure.AutoConfiguration.imports
com.example.sms.SmsAutoConfiguration

// Step 4: Users configure via application.properties
# sms.api-key=ABC123
# sms.sender-id=MyApp
```

CHAPTER 3

Spring Boot Configuration In-Depth

Properties, YAML, Profiles, @ConfigurationProperties & Externalized Config

3.1 application.properties vs application.yml

Feature	application.properties	application.yml
Format	Key=value flat format	Hierarchical YAML
Readability	Verbose for nested keys	Clean hierarchy with indentation
Lists	server.allowed[0]=a, [1]=b	server:\n allowed:\n - a\n - b
Profiles	application-dev.properties	--- with spring.config.activate.on-profile
Preference	Simple apps	Complex configs with nesting

■ application.properties

```
# application.properties
server.port=8080
server.servlet.context-path=/api

spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

logging.level.com.example=DEBUG
logging.level.org.springframework.web=INFO
```

■ application.yml

```
# application.yml – same config in YAML
server:
port: 8080
servlet:
context-path: /api

spring:
datasource:
url: jdbc:h2:mem:testdb
username: sa
password:
```

```
jpa:
hibernate:
ddl-auto: update
show-sql: true
properties:
hibernate:
format_sql: true

logging:
level:
com.example: DEBUG
org.springframework.web: INFO
```

3.2 @ConfigurationProperties

@ConfigurationProperties

@ConfigurationProperties binds a group of related properties to a strongly-typed Java class. This is the recommended way to handle custom application properties – it provides type safety, IDE auto-completion, and validation support.

■ Properties to bind

```
# application.properties
app.name=MySpringApp
app.max-retry=3
app.timeout-seconds=30
app.allowed-origins=http://localhost:3000,https://myfrontend.com
app.database.host=localhost
app.database.port=5432
```

■ @ConfigurationProperties binding

```
// Properties class
@ConfigurationProperties(prefix = "app")
@Component // or use @EnableConfigurationProperties(AppProperties.class)
public class AppProperties {
private String name;
private int maxRetry;
private int timeoutSeconds;
private List<String> allowedOrigins = new ArrayList<>();
private Database database = new Database();

// Getters and setters required

public static class Database {
private String host;
private int port;
// getters and setters
}
```

```
}

// Usage in a Service
@Service
public class StartupService {
private final AppProperties props;
public StartupService(AppProperties props) { this.props = props; }

public void logConfig() {
System.out.println("App: " + props.getName());
System.out.println("DB: " + props.getDatabase().getHost()
+ ":" + props.getDatabase().getPort());
}
}
```

3.3 Profiles

Spring Profile

A profile is a named logical grouping of configuration and beans. You activate profiles at startup to switch between environments (dev, test, prod) without changing code. Beans annotated with @Profile are only created when that profile is active.

■ **Profile-specific properties**

```
# application-dev.properties (active when spring.profiles.active=dev)
spring.datasource.url=jdbc:h2:mem:devdb
spring.jpa.show-sql=true
logging.level.com.example=DEBUG

# application-prod.properties (active when spring.profiles.active=prod)
spring.datasource.url=jdbc:postgresql://prodhost:5432/mydb
spring.datasource.username=produser
spring.datasource.password=${DB_PASSWORD}
spring.jpa.show-sql=false
logging.level.com.example=WARN

# Activate in application.properties
spring.profiles.active=dev

# Or at runtime via command line:
# java -jar myapp.jar --spring.profiles.active=prod
# Or via environment variable:
# SPRING_PROFILES_ACTIVE=prod
```

■ **@Profile on beans**

```
// @Profile on bean – only created in matching profile
@Configuration
public class DataSourceConfig {

@Bean
```

```
@Profile("dev")
public DataSource h2DataSource() {
return new EmbeddedDatabaseBuilder()
.setType(EmbeddedDatabaseType.H2).build();
}
@Bean
@Profile("prod")
public DataSource postgresDataSource() {
HikariConfig cfg = new HikariConfig();
cfg.setJdbcUrl("jdbc:postgresql://...");
return new HikariDataSource(cfg);
}
}
// @Profile on a whole @Service
@Service
@Profile("dev")
public class MockEmailService implements EmailService {
public void send(String to, String body) {
System.out.println("[MOCK] Email to " + to + ": " + body);
}
}
```

3.4 Externalized Configuration Priority Order

Spring Boot loads properties in this order (higher = overrides lower):

Priority	Source
1 (highest)	Command-line arguments: --server.port=9090
2	SPRING_APPLICATION_JSON environment variable (inline JSON)
3	OS Environment variables: SERVER_PORT=9090
4	System properties: -Dserver.port=9090
5	application-{profile}.properties outside JAR
6	application.properties outside JAR (next to JAR)
7	application-{profile}.properties inside JAR (classpath)
8 (lowest)	application.properties inside JAR (classpath)

CHAPTER 4

Web Applications in Spring Boot

Spring MVC, DispatcherServlet, Controllers, Request Mapping & Exception Handling

4.1 Spring MVC Architecture

Spring MVC

Spring MVC (Model-View-Controller) is the web framework included in spring-boot-starter-web. It processes HTTP requests through a central DispatcherServlet, routes them to @Controller handler methods, and renders responses via View Resolvers (for HTML) or MessageConverters (for JSON/XML REST APIs).

Request Flow:

- **1. Browser/Client** sends HTTP request.
- **2. DispatcherServlet** receives all requests (Front Controller pattern).
- **3. HandlerMapping** looks up which @Controller method matches the URL.
- **4. HandlerAdapter** invokes the matched controller method.
- **5. Controller** processes the request, calls services, returns Model+View name or response body.
- **6. ViewResolver** resolves the view name to a Thymeleaf/JSP template (for MVC apps).
- **7. MessageConverter** serialises Java objects to JSON/XML (for REST APIs).
- **8. Response** is sent back to the client.

4.2 @Controller vs @RestController

Annotation	Returns	Use Case
@Controller	View name (String) + Model	Traditional MVC – renders HTML templates
@RestController	Object → JSON/XML body	REST APIs – returns data, not views

■ @Controller vs @RestController

```
// @Controller – returns a Thymeleaf view
@Controller
public class PageController {

@GetMapping("/home")
public String home(Model model) {
model.addAttribute("message", "Welcome!");
return "home"; // resolves to templates/home.html
```

```
}
}

// @RestController – returns JSON
@RestController
@RequestMapping("/api/users")
public class UserController {

@GetMapping("/{id}")
public User getUser(@PathVariable Long id) {
return userService.findById(id); // Jackson converts to JSON
}
}
```

4.3 Request Mapping Annotations

Annotation	HTTP Method	Purpose
@RequestMapping	Any (specify method=)	General-purpose mapping
@GetMapping	GET	Retrieve resources
@PostMapping	POST	Create new resources
@PutMapping	PUT	Update/replace a resource
@PatchMapping	PATCH	Partially update a resource
@DeleteMapping	DELETE	Delete a resource

4.4 Method Parameters – Extracting Request Data

Annotation	Extracts From	Example
@PathVariable	URL path segment	@GetMapping("/{id}") + @PathVariable Long id
@RequestParam	Query string (?key=val)	@RequestParam(required=false) String name
@RequestBody	Request body (JSON)	@PostMapping + @RequestBody User user
@RequestHeader	HTTP header	@RequestHeader("Authorization") String token
@CookieValue	Cookie	@CookieValue("sessionId") String sessionId
@ModelAttribute	Form fields	@PostMapping + @ModelAttribute UserForm form

■ **Full CRUD Controller with all mapping annotations**

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

// GET /api/products/42
@GetMapping("/{id}")
```

```
public Product getById(@PathVariable Long id) {
return productService.findById(id);
}
// GET /api/products?category=electronics&minPrice;=100
@GetMapping
public List<Product> search(
@RequestParam String category,
@RequestParam(defaultValue = "0") double minPrice) {
return productService.search(category, minPrice);
}
// POST /api/products (body: {"name":"TV","price":999})
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Product create(@RequestBody @Valid Product product) {
return productService.save(product);
}
// PUT /api/products/42
@PutMapping("/{id}")
public Product update(@PathVariable Long id,
@RequestBody @Valid Product product) {
return productService.update(id, product);
}
// DELETE /api/products/42
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable Long id) {
productService.delete(id);
}
}
```

4.5 Exception Handling – @ControllerAdvice

@ControllerAdvice / @RestControllerAdvice

@ControllerAdvice is a specialised @Component that applies across all controllers globally. Combined with @ExceptionHandler methods, it provides a centralised place to catch exceptions and return consistent error responses instead of scattering try-catch blocks everywhere.

■ Global Exception Handling

```
// Custom exception
public class ResourceNotFoundException extends RuntimeException {
public ResourceNotFoundException(String msg) { super(msg); }
}
// Standardised error response body
```

```
public class ErrorResponse {
private int status;
private String message;
private LocalDateTime timestamp;
// constructor, getters...
}

// Global exception handler
@RestControllerAdvice
public class GlobalExceptionHandler {

@ExceptionHandler(ResourceNotFoundException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public ErrorResponse handleNotFound(ResourceNotFoundException ex) {
return new ErrorResponse(404, ex.getMessage(), LocalDateTime.now());
}

@ExceptionHandler(MethodArgumentNotValidException.class)
@ResponseStatus(HttpStatus.BAD_REQUEST)
public ErrorResponse handleValidation(MethodArgumentNotValidException ex) {
String msg = ex.getBindingResult().getFieldErrors().stream()
.map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
.collect(Collectors.joining(", "));
return new ErrorResponse(400, msg, LocalDateTime.now());
}

@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ErrorResponse handleGeneral(Exception ex) {
return new ErrorResponse(500, "Internal server error", LocalDateTime.now());
}
}
```

CHAPTER 5

Building REST APIs

Full CRUD API, ResponseEntity, Validation & File Upload

5.1 ResponseEntity – Full HTTP Control

ResponseEntity

ResponseEntity gives you complete control over the HTTP response – status code, headers, and body. It is the recommended return type for REST controllers when you need to return different status codes or custom headers.

■ Full REST CRUD with ResponseEntity

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

private final EmployeeService service;
public EmployeeController(EmployeeService service) { this.service = service; }

@GetMapping
public ResponseEntity<List<Employee>> getAll() {
return ResponseEntity.ok(service.findAll()); // 200 OK
}

@GetMapping("/{id}")
public ResponseEntity<Employee> getById(@PathVariable Long id) {
return service.findById(id)
.map(ResponseEntity::ok)
.orElse(ResponseEntity.notFound().build()); // 404
}

@PostMapping
public ResponseEntity<Employee> create(@RequestBody @Valid Employee emp) {
Employee saved = service.save(emp);
URI location = URI.create("/api/employees/" + saved.getId());
return ResponseEntity.created(location).body(saved); // 201 Created
}

@PutMapping("/{id}")
public ResponseEntity<Employee> update(
@PathVariable Long id,
@RequestBody @Valid Employee emp) {
if (!service.exists(id)) return ResponseEntity.notFound().build();
```

```
emp.setId(id);
return ResponseEntity.ok(service.save(emp));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
if (!service.exists(id)) return ResponseEntity.notFound().build();
service.delete(id);
return ResponseEntity.noContent().build(); // 204 No Content
}
}
```

5.2 Bean Validation

Bean Validation (Jakarta Validation)

Bean Validation (formerly javax.validation) allows you to declare constraints on Java class fields using annotations. When you annotate a controller parameter with @Valid, Spring automatically validates the object and throws MethodArgumentNotValidException if it fails.

Annotation	Description
@NotNull	Field must not be null
@NotBlank	String must not be null, empty, or whitespace
@NotEmpty	String/collection must not be null or empty
@Size(min, max)	String/collection size must be within range
@Min(value)	Number must be >= value
@Max(value)	Number must be <= value
@Email	String must be a valid email address
@Pattern(regexp)	String must match the regex pattern
@Positive	Number must be > 0
@Future	Date must be in the future
@Past	Date must be in the past

Bean Validation on Entity & Controller

```
// Entity with validation annotations
public class Employee {
private Long id;

@NotBlank(message = "Name is required")
@Size(min = 2, max = 50, message = "Name must be 2-50 characters")
private String name;

@NotBlank
```

```
@Email(message = "Must be a valid email")
private String email;

@NotNull
@Min(value = 1000, message = "Salary must be at least 1000")
private Double salary;

@NotBlank
@Pattern(regexp = "IT|HR|FINANCE", message = "Invalid department")
private String department;
}

// Controller uses @Valid to trigger validation
@PostMapping
public ResponseEntity<Employee> create(@RequestBody @Valid Employee emp) {
return ResponseEntity.created(...).body(service.save(emp));
}
```

5.3 HTTP Status Code Reference

Status	Meaning	When to Use
200 OK	Success	GET / PUT / PATCH success
201 Created	Resource created	POST that creates a resource
204 No Content	Success, no body	DELETE success
400 Bad Request	Invalid request/body	Validation failure
401 Unauthorized	Authentication required	No/invalid credentials
403 Forbidden	Access denied	Not authorised
404 Not Found	Resource not found	Resource doesn't exist
409 Conflict	State conflict	Duplicate resource
422 Unprocessable	Semantic validation fail	Business rule violation
500 Internal Server Error	Unexpected server error	Unhandled exceptions

CHAPTER 6

Spring Boot Data & Persistence

Auto-configured DataSource, JPA, H2 Console & Repository Pattern

6.1 Spring Boot + Spring Data JPA

Spring Boot JPA Auto-Configuration

When spring-boot-starter-data-jpa is on the classpath, Spring Boot automatically: configures a DataSource from spring.datasource.* properties, creates a Hibernate EntityManagerFactory, sets up JPA transaction management, and enables Spring Data repository scanning. You only need to write your @Entity and JpaRepository interface.

■ JPA Configuration in application.properties

```
# application.properties – all JPA needs
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Hibernate
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true

# H2 browser console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

■ Complete Entity + Repository + Service

```
// Entity
@Entity
@Table(name = "employees")
public class Employee {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String name;
private String email;
private double salary;
// constructors, getters, setters
}

// Repository – Spring Boot auto-scans and implements this
```

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
List<Employee> findByNameContainingIgnoreCase(String name);
Optional<Employee> findByEmail(String email);
}

// Service
@Service
@Transactional
public class EmployeeService {
private final EmployeeRepository repo;
public EmployeeService(EmployeeRepository repo) { this.repo = repo; }

public Employee save(Employee e) { return repo.save(e); }
public List<Employee> findAll() { return repo.findAll(); }
public Optional<Employee> findById(Long id) { return repo.findById(id); }
public void delete(Long id) { repo.deleteById(id); }
}
```

6.2 Database Initialisation Scripts

- **schema.sql** – Place in src/main/resources; runs DDL on startup (CREATE TABLE etc).
- **data.sql** – Place in src/main/resources; runs DML on startup (INSERT etc).
- Controlled by **spring.sql.init.mode=always/embedded/never**.

■ schema.sql & data.sql

```
-- src/main/resources/schema.sql
CREATE TABLE IF NOT EXISTS employees (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL,
salary DOUBLE NOT NULL
);

-- src/main/resources/data.sql
INSERT INTO employees (name, email, salary) VALUES
('Alice', 'alice@example.com', 75000),
('Bob', 'bob@example.com', 82000);
```

CHAPTER 7

Spring Boot Actuator

Production Monitoring – Health, Metrics, Info & Custom Indicators

7.1 What is Actuator?

Spring Boot Actuator

Actuator adds production-ready monitoring and management endpoints to your Spring Boot application over HTTP or JMX. It exposes information about the application's health, metrics, environment, beans, mappings, and more – without writing any code.

Actuator Setup

```
<!-- pom.xml -->
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

# application.properties – expose all endpoints
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
management.server.port=8081 # separate port for security
```

7.2 Built-in Endpoints

Endpoint URL	Method	Description
/actuator/health	GET	App health status (UP/DOWN) + component details
/actuator/info	GET	Custom app info (name, version, git commit)
/actuator/metrics	GET	List of available metrics
/actuator/metrics/{name}	GET	Details for a specific metric
/actuator/env	GET	All environment properties and active profiles
/actuator/beans	GET	All Spring beans in the application context
/actuator/mappings	GET	All @RequestMapping URLs and handler methods
/actuator/loggers	GET	Current logging levels; POST to change at runtime
/actuator/threaddump	GET	Current thread dump
/actuator/heapdump	GET	Download a heap dump file

/actuator/shutdown	POST	Gracefully shutdown the app (disabled by default)

7.3 Custom Health Indicator

Custom HealthIndicator

```
@Component
public class ExternalApiHealthIndicator implements HealthIndicator {
private final RestTemplate restTemplate;
public ExternalApiHealthIndicator(RestTemplate rt) { this.restTemplate = rt; }
@Override
public Health health() {
try {
ResponseEntity<String> resp =
restTemplate.getForEntity("https://api.example.com/ping", String.class);
if (resp.getStatusCode().is2xxSuccessful()) {
return Health.up()
.withDetail("status", "External API reachable")
.build();
}
return Health.down().withDetail("status", resp.getStatusCode()).build();
} catch (Exception ex) {
return Health.down(ex).withDetail("error", ex.getMessage()).build();
}
}
}
```

CHAPTER 8

Spring Boot Testing

@SpringBootTest, @WebMvcTest, MockMvc & TestRestTemplate

8.1 Testing Annotations

Annotation / Class	Purpose
@SpringBootTest	Loads full application context; integration test
@WebMvcTest	Loads only web layer (controllers); unit test
@DataJpaTest	Loads only JPA layer; in-memory DB; repository test
@MockBean	Adds a Mockito mock to the Spring context
@SpyBean	Adds a Mockito spy wrapping a real bean
MockMvc	Performs HTTP requests against controllers without server
TestRestTemplate	Performs real HTTP requests to running server port
@AutoConfigureMockMvc	Auto-wires MockMvc in @SpringBootTest

8.2 @WebMvcTest – Controller Unit Test

■ @WebMvcTest with MockMvc

```
@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {

@Autowired
private MockMvc mockMvc;

@MockBean
private EmployeeService service; // mock the service layer

@Autowired
private ObjectMapper objectMapper;

@Test
void getById_shouldReturnEmployee() throws Exception {
Employee emp = new Employee(1L, "Alice", "alice@test.com", 70000);
given(service.findById(1L)).willReturn(Optional.of(emp));

mockMvc.perform(get("/api/employees/1")
.contentType(MediaType.APPLICATION_JSON))
.andExpect(status().isOk())
```

```
.andExpect(jsonPath("$.name").value("Alice"))
.andExpect(jsonPath("$.email").value("alice@test.com"));
}

@Test
void create_shouldReturn201() throws Exception {
Employee emp = new Employee(null, "Bob", "bob@test.com", 60000);
Employee saved = new Employee(2L, "Bob", "bob@test.com", 60000);
given(service.save(any())).willReturn(saved);

mockMvc.perform(post("/api/employees")
.contentType(MediaType.APPLICATION_JSON)
.content(objectMapper.writeValueAsString(emp)))
.andExpect(status().isCreated())
.andExpect(jsonPath("$.id").value(2));
}

@Test
void getById_notFound_shouldReturn404() throws Exception {
given(service.findById(999L)).willReturn(Optional.empty());

mockMvc.perform(get("/api/employees/999"))
.andExpect(status().isNotFound());
}
}
```

8.3 @SpringBootTest – Integration Test

■ @SpringBootTest Integration Test

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class EmployeeIntegrationTest {

@Autowired
private TestRestTemplate restTemplate;

@Autowired
private EmployeeRepository repo;

@BeforeEach
void setup() {
repo.deleteAll();
repo.save(new Employee(null, "Alice", "alice@test.com", 75000));
}

@Test
void getAll_shouldReturnEmployees() {
ResponseEntity<Employee[]> resp =
restTemplate.getForEntity("/api/employees", Employee[].class);
assertThat(resp.getStatusCode()).isEqualTo(HttpStatus.OK);
```

```
    assertThat(resp.getBody()).hasSize(1);
    assertThat(resp.getBody()[0].getName()).isEqualTo("Alice");
    }

    @Test
    void create_shouldPersist() {
    Employee emp = new Employee(null, "Bob", "bob@test.com", 60000);
    ResponseEntity<Employee> resp =
    restTemplate.postForEntity("/api/employees", emp, Employee.class);
    assertThat(resp.getStatusCode()).isEqualTo(HttpStatus.CREATED);
    assertThat(repo.count()).isEqualTo(2);
    }
    }
```

Quick Reference – Spring Boot Annotations

Annotation	Layer	Purpose
@SpringBootApplication	Main	Bootstrap: @Configuration + @EnableAutoConfiguration + @ComponentScan
@RestController	Web	JSON/XML REST controller (@Controller + @ResponseBody)
@RequestMapping	Web	Maps URL patterns to a controller or method
@GetMapping / @PostMapping etc.	Web	Shortcut HTTP method mapping annotations
@PathVariable	Web	Binds URL path segment to method parameter
@RequestParam	Web	Binds query parameter to method parameter
@RequestBody	Web	Deserialises JSON request body to Java object
@ResponseStatus	Web	Sets default HTTP status code for a method
@ControllerAdvice	Web	Global exception handling across all controllers
@ExceptionHandler	Web	Handles a specific exception type in advice
@Valid / @Validated	Web	Triggers Bean Validation on method argument
@ConfigurationProperties	Config	Binds group of properties to a POJO
@Profile	Config	Activates bean only for specified profile
@Value	Config	Injects a single property value
@ConditionalOnClass	Auto-config	Register bean only if class is on classpath
@ConditionalOnMissingBean	Auto-config	Register bean only if no bean of type exists
@ConditionalOnProperty	Auto-config	Register bean based on property value
@SpringBootTest	Test	Loads full application context for integration tests
@WebMvcTest	Test	Loads only web layer for controller unit tests
@MockBean	Test	Replaces a bean with a Mockito mock in context

✔ Spring Boot 3.x requires Java 17+ and uses Jakarta EE 10 (jakarta.* packages instead of javax.*). Always use spring-boot-starter-parent as your parent POM to get managed dependency versions.

■ Never expose all Actuator endpoints in production without authentication. Use management.endpoints.web.exposure.include to whitelist only what you need, and secure them with Spring Security.